

Mini RFC: Connection ID

April 5, 2022

Note: This is a simplified and annotated summary of the QUIC-related IETF RFCs. It's part of a course project at Tsinghua University and not suitable for any other purposes.

We reorder paragraphs in QUIC RFCs and omit many details to improve its readability for beginners.

WARNING: DO NOT distribute this document. Modification of extracts from IETF RFCs is not granted by IETF as original authors retain copyrights of RFCs. This document is for educational purposes only.

1 Overview

Each connection possesses a set of connection identifiers, or connection IDs, each of which can identify the connection. Connection IDs are independently selected by endpoints; each endpoint selects the connection IDs that its **peer** uses.

The primary function of a connection ID is to ensure that changes in addressing at lower protocol layers (UDP, IP) do not cause packets for a QUIC connection to be delivered to the wrong endpoint. Each endpoint selects connection IDs using an implementation-specific (and perhaps deployment-specific) method that will allow packets with that connection ID to be routed back to the endpoint and to be identified by the endpoint upon receipt.

Multiple connection IDs are used so that endpoints can send packets that cannot be identified by an observer as being for the same connection without cooperation from an endpoint.

Connection IDs **MUST NOT** contain any information that can be used by an external observer (that is, one that does not cooperate with the issuer) to correlate them with other connection IDs for the same connection. As a trivial example, this means the same connection ID **MUST NOT** be issued more than once on the same connection.

Packets with long headers include Source Connection ID and Destination Connection ID fields. These fields are used to set the connection IDs for new connections.

Packets with short headers only include the Destination Connection ID and omit the explicit length. The length of the Destination Connection ID field is expected to be known to endpoints. A zero-length connection ID can

Connection ID is used to identify a incoming connection. In TCP, we use the well-known "five tuple", <source IP, source port, destination IP, destination port, protocol> as a connection identifier.

What we did in Project I.

be used when a connection ID is not needed to route to the correct endpoint. However, multiplexing connections on the same local IP address and port while using zero-length connection IDs will cause failures in the presence of peer connection migration, NAT rebinding, and client port reuse.

When an endpoint uses a non-zero-length connection ID, it needs to ensure that the peer has a supply of connection IDs from which to choose for packets sent to the endpoint. These connection IDs are supplied by the endpoint using the `NEW_CONNECTION_ID` frame.

2 Connection ID Negotiation

During the handshake, packets with the long header are used to establish the connection IDs used by both endpoints. Each endpoint uses the Source Connection ID field to specify the connection ID that is used in the Destination Connection ID field of packets being sent to them. After processing the first Initial packet, each endpoint sets the Destination Connection ID field in subsequent packets it sends to the value of the Source Connection ID field that it received.

When an Initial packet is sent by a client that has not previously received an Initial or Retry packet from the server, the client populates the Destination Connection ID field with an unpredictable value. This Destination Connection ID **MUST** be at least 8 bytes in length. Until a packet is received from the server, the client **MUST** use the same Destination Connection ID value on all packets in this connection.

The client populates the Source Connection ID field with a value of its choosing and sets the Source Connection ID Length field to indicate the length. Upon first receiving an Initial or Retry packet from the server, the client uses the Source Connection ID supplied by the server as the Destination Connection ID for subsequent packets, including any 0-RTT packets. This means that a client might have to change the connection ID it sets in the Destination Connection ID field twice during connection establishment: once in response to a Retry packet and once in response to an Initial packet from the server. Once a client has received a valid Initial packet from the server, it **MUST** discard any subsequent packet it receives on that connection with a different Source Connection ID.

A client **MUST** change the Destination Connection ID it uses for sending packets in response to only the first received Initial or Retry packet. A server **MUST** set the Destination Connection ID it uses for sending packets based on the first received Initial packet. Any further changes to the Destination Connection ID are only permitted if the values are taken from `NEW_CONNECTION_ID` frames; if subsequent Initial packets include a different Source Connection ID, they **MUST** be discarded. This avoids unpredictable outcomes that might otherwise result from stateless processing of multiple Initial packets with different Source Connection IDs.

Consider the case that a client sends two Initial packets due to network delay, the server should know that these two packets are of the same connection from the fact that they select the same fake Destination ID. Otherwise the server will treat them as two individual connection initialization requests and create two connections.

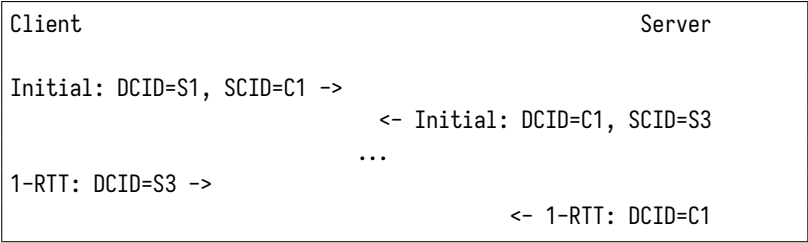


Figure 1: Use of Connection IDs in a Handshake

3 Issuing Connection ID

Each connection ID has an associated sequence number to assist in detecting when NEW_CONNECTION_ID or RETIRE_CONNECTION_ID frames refer to the same value. The initial connection ID issued by an endpoint is sent in the Source Connection ID field of the long packet header during the handshake. The sequence number of the initial connection ID is 0.

Additional connection IDs are communicated to the peer using NEW_CONNECTION_ID frames. The sequence number on each newly issued connection ID MUST increase by 1. The connection ID that a client selects for the first Destination Connection ID field it sends are not assigned sequence numbers.

When an endpoint issues a connection ID, it MUST accept packets that carry this connection ID for the duration of the connection or until its peer invalidates the connection ID via a RETIRE_CONNECTION_ID frame. Connection IDs that are issued and not retired are considered active; any active connection ID is valid for use with the current connection at any time, in any packet type. This includes the connection ID issued by the server via the preferred_address transport parameter.

An endpoint SHOULD ensure that its peer has a sufficient number of available and unused connection IDs. Endpoints advertise the number of active connection IDs they are willing to maintain using the active_connection_id_limit transport parameter. An endpoint MUST NOT provide more connection IDs than the peer's limit. An endpoint MAY send connection IDs that temporarily exceed a peer's limit if the NEW_CONNECTION_ID frame also requires the retirement of any excess, by including a sufficiently large value in the Retire Prior To field.

A NEW_CONNECTION_ID frame might cause an endpoint to add some active connection IDs and retire others based on the value of the Retire Prior To field. After processing a NEW_CONNECTION_ID frame and adding and retiring active connection IDs, if the number of active connection IDs exceeds the value advertised in its active_connection_id_limit transport parameter, an endpoint MUST close the connection with an error of type CONNECTION_ID_LIMIT_ERROR.

An endpoint SHOULD supply a new connection ID when the peer retires a connection ID. If an endpoint provided fewer connection IDs than

Use a default of 2 until you implement Transport Parameter Exchange (5 pt)

Red color means error handling rules, you can ignore them unless you implement the Error Handling (10 pt) feature.

the peer's `active_connection_id_limit`, it MAY supply a new connection ID when it receives a packet with a previously unused connection ID. An endpoint MAY limit the total number of connection IDs issued for each connection to avoid the risk of running out of connection IDs. An endpoint MAY also limit the issuance of connection IDs to reduce the amount of per-path state it maintains, such as path validation status, as its peer might interact with it over as many paths as there are issued connection IDs.

An endpoint that initiates migration and requires non-zero-length connection IDs SHOULD ensure that the pool of connection IDs available to its peer allows the peer to use a new connection ID on migration, as the peer will be unable to respond if the pool is exhausted.

An endpoint that selects a zero-length connection ID during the handshake cannot issue a new connection ID. A zero-length Destination Connection ID field is used in all packets sent toward such an endpoint over any network path.

You are free to choose the connection issuance policy and could issue a ID whenever the specification allows as we don't open many connections in our test cases.

4 Consuming and Retiring Connection IDs

An endpoint can change the connection ID it uses for a peer to another available one at any time during the connection. [An endpoint consumes connection IDs in response to a migrating peer.](#)

An endpoint maintains a set of connection IDs received from its peer, any of which it can use when sending packets. When the endpoint wishes to remove a connection ID from use, it sends a `RETIRE_CONNECTION_ID` frame to its peer. Sending a `RETIRE_CONNECTION_ID` frame indicates that the connection ID will not be used again and requests that the peer replace it with a new connection ID using a `NEW_CONNECTION_ID` frame.

Endpoints limit the use of a connection ID to packets sent from a single local address to a single destination address. Endpoints SHOULD retire connection IDs when they are no longer actively using either the local or destination address for which the connection ID was used.

An endpoint might need to stop accepting previously issued connection IDs in certain circumstances. Such an endpoint can cause its peer to retire connection IDs by sending a `NEW_CONNECTION_ID` frame with an increased `Retire Prior To` field. The endpoint SHOULD continue to accept the previously issued connection IDs until they are retired by the peer. If the endpoint can no longer process the indicated connection IDs, it MAY close the connection.

Upon receipt of an increased `Retire Prior To` field, the peer MUST stop using the corresponding connection IDs and retire them with `RETIRE_CONNECTION_ID` frames before adding the newly provided connection ID to the set of active connection IDs. This ordering allows an endpoint to replace all active connection IDs without the possibility of a peer having no available connection IDs and without exceeding the limit the peer sets in the `active_connection_id_limit` transport parameter. Failure to cease using the connection IDs when requested can result in connection failures, as the issuing endpoint might be

Related optional feature:
Connection Migration (10 pt)

You cannot make it possible for an external observer to link two QUIC connections from different network addresses by their QUIC connection IDs.

unable to continue using the connection IDs with the active connection.

An endpoint SHOULD limit the number of connection IDs it has retired locally for which RETIRE_CONNECTION_ID frames have not yet been acknowledged. An endpoint SHOULD allow for sending and tracking a number of RETIRE_CONNECTION_ID frames of at least twice the value of the active_connection_id_limit transport parameter. An endpoint MUST NOT forget a connection ID without retiring it, though it MAY choose to treat having connection IDs in need of retirement that exceed this limit as a connection error of type CONNECTION_ID_LIMIT_ERROR.

Endpoints SHOULD NOT issue updates of the Retire Prior To field before receiving RETIRE_CONNECTION_ID frames that retire all connection IDs indicated by the previous Retire Prior To value.

5 Packets and Frames

5.1 Packets

There is no new type of packets introduced in this RFC.

5.2 Frames

5.2.1 NEW_CONNECTION_ID Frames

An endpoint sends a NEW_CONNECTION_ID frame (type=0x18) to provide its peer with alternative connection IDs that can be used to break linkability when migrating connections.

```
NEW_CONNECTION_ID Frame {  
  Type (i) = 0x18,  
  Sequence Number (i),  
  Retire Prior To (i),  
  Length (8),  
  Connection ID (8..160),  
  Stateless Reset Token (128),  
}
```

Figure 2: *NEW_CONNECTION_ID Frame Format*

- **Sequence Number:** The sequence number assigned to the connection ID by the sender, encoded as a variable-length integer.
- **Retire Prior To:** A variable-length integer indicating which connection IDs should be retired.
- **Length:** An 8-bit unsigned integer containing the length of the connection ID. Values less than 1 and greater than 20 are invalid.

- **Connection ID:** A connection ID of the specified length.
- **Stateless Reset Token:** A 128-bit value that will be used for a stateless reset when the associated connection ID is used.

Related Optional Feature: *Stateless Reset* (5pt)

An endpoint **MUST NOT** send this frame if it currently requires that its peer send packets with a zero-length Destination Connection ID. Changing the length of a connection ID to or from zero length makes it difficult to identify when the value of the connection ID changed. **An endpoint that is sending packets with a zero-length Destination Connection ID *MUST* treat receipt of a NEW_CONNECTION_ID frame as a connection error of type PROTOCOL_VIOLATION.**

Transmission errors, timeouts, and retransmissions might cause the same NEW_CONNECTION_ID frame to be received multiple times. Receipt of the same frame multiple times **MUST NOT** be treated as a connection error. A receiver can use the sequence number supplied in the NEW_CONNECTION_ID frame to handle receiving the same NEW_CONNECTION_ID frame multiple times.

If an endpoint receives a NEW_CONNECTION_ID frame that repeats a previously issued connection ID with a different Stateless Reset Token field value or a different Sequence Number field value, or if a sequence number is used for different connection IDs, the endpoint *MAY* treat that receipt as a connection error of type PROTOCOL_VIOLATION.

Related optional features: *Address Validation* (5pt)

The Retire Prior To field applies to connection IDs established during connection setup. The value in the Retire Prior To field **MUST** be less than or equal to the value in the Sequence Number field. **Receiving a value in the Retire Prior To field that is greater than that in the Sequence Number field *MUST* be treated as a connection error of type FRAME_ENCODING_ERROR.**

Once a sender indicates a Retire Prior To value, smaller values sent in subsequent NEW_CONNECTION_ID frames have no effect. A receiver **MUST** ignore any Retire Prior To fields that do not increase the largest received Retire Prior To value.

An endpoint that receives a NEW_CONNECTION_ID frame with a sequence number smaller than the Retire Prior To field of a previously received NEW_CONNECTION_ID frame **MUST** send a corresponding RETIRE_CONNECTION_ID frame that retires the newly received connection ID, unless it has already done so for that sequence number.

5.2.2 RETIRE_CONNECTION_ID Frames

An endpoint sends a RETIRE_CONNECTION_ID frame (type=0x19) to indicate that it will no longer use a connection ID that was issued by its peer. This includes the connection ID provided during the handshake. Sending a RETIRE_CONNECTION_ID frame also serves as a request to the peer to send additional connection IDs for future use. New connection IDs can be delivered to a peer using the NEW_CONNECTION_ID frame.

- **Sequence Number:** The sequence number of the connection ID being retired.

```
RETIRE_CONNECTION_ID Frame {  
    Type (i) = 0x19,  
    Sequence Number (i),  
}
```

Figure 3: *RETIRE_CONNECTION_ID Frame Format*

Receipt of a RETIRE_CONNECTION_ID frame containing a sequence number greater than any previously sent to the peer MUST be treated as a connection error of type PROTOCOL_VIOLATION.

The sequence number specified in a RETIRE_CONNECTION_ID frame MUST NOT refer to the Destination Connection ID field of the packet in which the frame is contained. The peer MAY treat this as a connection error of type PROTOCOL_VIOLATION.

An endpoint cannot send this frame if it was provided with a zero-length connection ID by its peer. An endpoint that provides a zero-length connection ID MUST treat receipt of a RETIRE_CONNECTION_ID frame as a connection error of type PROTOCOL_VIOLATION.

6 Application Protocol

There is no new application protocol defined in this mini RFC.